

Welcome back

How was lab? Was anything confusing?

Review

We covered:

- Jupyter as a powerful calculator
- Operations: addition, subtraction, multiplication, division (two types of division!)
- Special operations: exponentiation, modulus
- Errors
- Comments

$$\begin{array}{ccccccc} \# & & 2 & & & & 8 \\ 2 & + & (3 * 4 // 5) & - & 2 ** (3 \% 10) \\ -4 & & & & & & \end{array}$$
 $1 +$

Cell In[2], line 1

 $1 + \wedge$

```
SyntaxError: invalid syntax
```

 $2 * * 5$

Cell In[10], line 1

$$2 * * 5$$

SyntaxError: invalid syntax

Text cells vs code cells - getting out of the text cell

```
# (-4) ** 0.5
```

$$(-4)^{**} 0.5$$

(1.2246467991473532e-16+2j)

0.00000000000000000000000001

1e-20

-5 % -4

```
-1
(-3) % 5
2
-3 // 5
-3 - (-1) * 5
342 // 12 + 23 ** 1 + 23 - 762
32 - 762
9 + 23
```

Reminder: if you're confused, you can test in Jupyter!

```
# Things that were confusing from earlier today
```

Beyond numbers: strings

Beyond numbers (like `1`), there are also other kinds of "things" in Python. The first one we'll cover are *strings*.

Strings are sequences of individual *characters*. Roughly, a character is a single piece of text. Don't worry too much about the definition of characters for now.

In Python, you can define a string by enclosing a piece of text in `"`. We'll see some examples

```
"Hello AddisCoder!"
'Hello AddisCoder!'
"hello addiscoder"
'hello addiscoder'
"678"
'678'
"True"
'True'
```

Beyond numbers: booleans

Aside from numbers and strings, there is another "thing" in Python called a *boolean*. A boolean is a True or False.

The syntax for defining a boolean is to use either `True` or `False`.

```
True
```

```
True
```

```
False
```

```
False
```

```
true
```

```
-----  
-----
```

```
NameError                                Traceback (most recent call  
last)
```

```
Cell In[64], line 1
```

```
----> 1 true
```

```
NameError: name 'true' is not defined
```

Putting it together: types

Each of the "things" we've covered has a *type*. Every "thing" in Python has a type.

You can determine the type of something by doing the following:

```
type(1)
```

```
int
```

```
type(1.0)
```

```
float
```

```
type("hello")
```

```
str
```

```
type(True)
```

```
bool
```

```
type(1 + 1)
```

[illegible]

```
Cell In[96], line 1
----> 1 int("1a")
```

ValueError: invalid literal for int() with base 10: '1a'

```
type("1.2")
```

```
str
```

```
int("1.2")
```

```
-----
-----
ValueError                                Traceback (most recent call
last)
```

```
Cell In[98], line 1
----> 1 int("1.2")
```

ValueError: invalid literal for int() with base 10: '1.2'

```
int("hello")
```

```
-----
-----
ValueError                                Traceback (most recent call
last)
```

```
Cell In[106], line 1
----> 1 int("hello")
```

ValueError: invalid literal for int() with base 10: 'hello'

```
int(True)
```

```
1
```

```
float(1)
```

```
1.0
```

```
1.2
```

```
1.2
```

```
float("1.2")
```

```
1.2
```

```
float(False)
```

```
0.0
```

```
float("1")
```

```
1.0
```

```
str(1)
'1'
str(1.2)
'1.2'
str(True)
'True'
bool(1)
True
bool(2)
True
bool(-100)
True
bool(0)
False
bool(0.01)
True
bool("true")
True
bool("false")
True
bool("")
False
bool("0.0")
True
bool(0.0)
False
bool(" ")
True
```

```
bool("0")
```

```
True
```

```
bool("False")
```

```
True
```

Operations that (can) convert between types

Beyond the arithmetic operations we've seen, there are also other kinds of operations. We'll start with equals and not equals. These operations can *change the types* of the things they operate on

```
type(1 + 1.0)
```

```
float
```

```
1 == 1
```

```
True
```

```
1 == 2
```

```
False
```

```
True == True
```

```
True
```

```
True == False
```

```
False
```

```
1 = = 1
```

```
Cell In[184], line 1
```

```
1 = = 1  
  ^
```

```
SyntaxError: cannot assign to literal
```

```
"hello" == "hello"
```

```
True
```

```
"hello" == "Hello"
```

```
False
```

```
"hello" == "hello "
```

False

1 != 1

False

1 != 2

True

1 ! = 2

Cell In[196], line 1

1 ! = 2
^

SyntaxError: invalid syntax

True != True

False

True != False

True

1 = 1

Cell In[202], line 1

1 = 1
^

SyntaxError: cannot assign to literal here. Maybe you meant '==' instead of '='?

1 == True

True

2 == True

False

bool(2)

True

0 == False

True

"" == False

False

```
int(True)
```

```
1
```

Heirachy of equality checking

- String
- Float
- Int
- Bool

```
" " == False
# Strings are not compatible with bools
# Automatically evaluates to false
```

```
False
```

```
" " == True
```

```
False
```

```
"1.0" == 1.0
```

```
False
```

```
1 == True
```

```
True
```

```
True == "a"
```

```
False
```

```
True == "True"
```

```
False
```

Explore other kinds of equals and not equals in lab!

Inequalities

You can also compare numbers

```
1 > 1
```

```
False
```

```
1 >= 1
```

```
True
```

```
1 >= 1
```

```
Cell In[236], line 1
```

```
1 >= 1
    ^
```

```
SyntaxError: invalid syntax
```

```
2 > 1
```

```
True
```

```
2 >= 1
```

```
True
```

```
2 < 1
```

```
False
```

```
2 <= 1
```

```
False
```

You can also compare things like strings and booleans. Explore what happens in lab!

Arithmetic for non-number types

Python also allows for arithmetic operations for non-number types. This can be very confusing, so let's walk through it slowly.

```
"Hello" + "AddisCoder!"
```

```
'HelloAddisCoder!'
```

```
"Hello" + " AddisCoder!"
```

```
'Hello AddisCoder!'
```

```
"Hello" + 1
```

```
-----
-----
TypeError                                Traceback (most recent call
last)
Cell In[250], line 1
----> 1 "Hello" + 1
```

```
TypeError: can only concatenate str (not "int") to str
```

```
"hello " * 5
"hello " + "hello " + "hello " + "hello " + "hello "
```

```
'hello hello hello hello hello '
```

```
"hello " * 0
```

```
''
```

```
"hello" // 3
```

```
-----
-----
TypeError                                Traceback (most recent call
last)
Cell In[262], line 1
----> 1 "hello" // 3
```

```
TypeError: unsupported operand type(s) for //: 'str' and 'int'
```

```
int(True)
```

```
1
```

```
1 + True
```

```
2
```

```
int(False)
```

```
0
```

```
1 + False
```

```
1
```

```
1 * True
```

```
1
```

```
1 * False
```

```
0
```

```
"hello " ** 2
```

```
-----
-----
TypeError                                Traceback (most recent call
last)
Cell In[276], line 1
----> 1 "hello " ** 2
```

```
TypeError: unsupported operand type(s) for ** or pow(): 'str' and 'int'
```

Advanced topic: variables

We will briefly cover an advanced topic called *variables*. Don't worry if this is confusing - you do not need understand them today. We will cover it in more detail tomorrow.

You've seen variables in math, like in $f(x)=x^2$, x is a variable.

A PROGRAMMING VARIABLE IS NOT A MATH VARIABLE!

A PROGRAMMING VARIABLE IS NOT A MATH VARIABLE!

So what is a variable

A programming variable "holds" some value. This value can be any of the types we've seen so far (they can also be other things, but we'll see this later).

The value of a programming variable can change! This is NOT TRUE for math variables!

```
x = 1
x = 1
x
1

x = 1
x = 2
x
2

x = 1
x = x + 1
x
2

x = "hello"
x
'hello'

x = True
x
```

True

```
x = 1
y = 2
x + y
```

3

```
x = 1
y = 2
z = x + y
z
```

3

```
x = 1
y = x
y
```

1

```
x = "hi"
y = "bye"
x + y
```

'hibye'

```
"hi" + "bye"
```

'hibye'